KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

제어문 1
조건문 if

조건에 따라
다른 줄을 실행하기

```
if total >= 20000:
    print("10% 할인")
elif total >= 10000:
    print("5% 할인")
else:
    print("할인 없음")
```

오늘의 순서

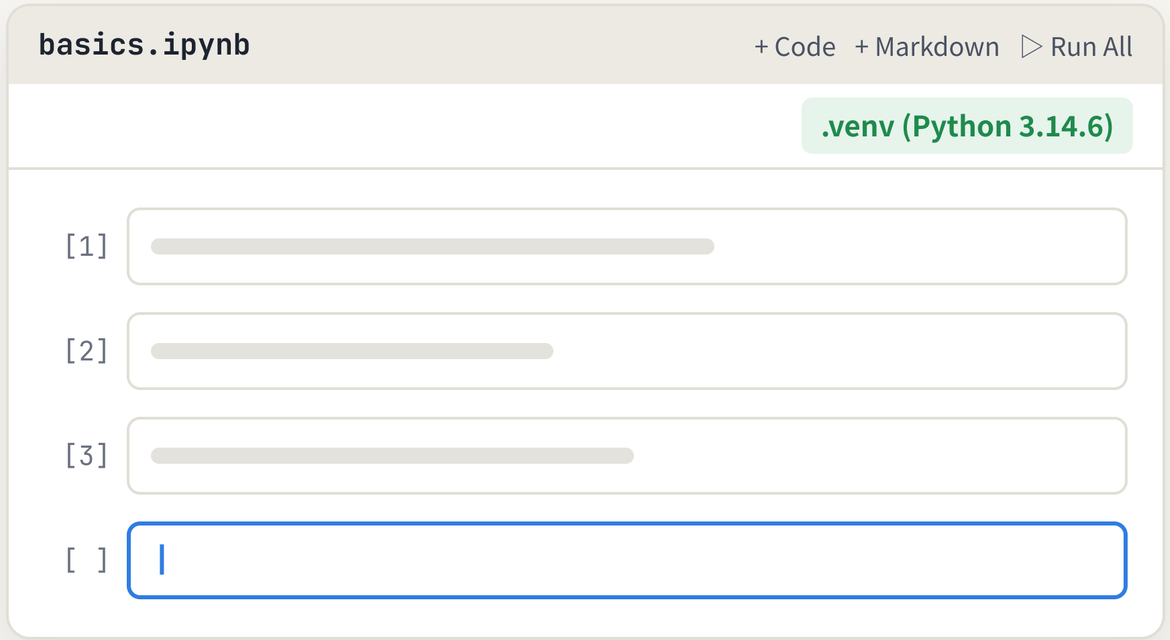
1	**참과 거짓으로 길 고르기** 왜 필요한가, 갈림길 그림, 가장 짧은 if, 조건은 True와 False	4–7쪽
2	**if, else, elif** False면 건너뛰기, 들여쓰기 범위, else, elif, 순서, if 여러 개	8–13쪽
3	**조건 합치기** and, if 안의 if, or, not in, 범위, 빈 값은 False	14–20쪽
4	**for와 함께, 줄여 쓰기** 골라 담기, 세기, items(), 한 줄 if else, get과 16 코드 다시 읽기	21–26쪽
5	**자주 나는 오류** = 와 ==, 콜론과 들여쓰기, or 함정, 글자와 숫자 비교	27–30쪽
6	**실습** 장바구니 할인, 성적 등급, 메뉴판 없는 주문 빼기, 모양 고르기	31–34쪽

3쪽은 노트북 준비, 35–36쪽은 막힐 때와 한 장 요약이에요.

들어가기

16에서 쓰던 노트북에 이어서 적어요

가져오는 것	다시 볼 곳
hello-uv 노트북, 커널 .venv	09 · 25–27쪽
비교 ==, >=, and, or, not	12 · 3파트
리스트, for, f-string	13
in과 not in	14
딕셔너리 menu와 get	15
집합과 discard	16



커널이 .venv인지 보고, 새 셀에서 Shift+Enter로 실행해요

1. 참과 거짓으로 길 고르기

메뉴판에 없는 주문이 오면 셀이 멈춰요

[1]
```
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
order = "tea"
print(menu[order])   ← tea가 없어도 이 줄을 실행해요
```
Python

```
KeyError: 'tea'
```

15, 16에서 읽기만 한 코드

```
menu["cakee"] if "cakee" in menu
    else 5000
```

```
if "juice" in menus:
    menus.remove("juice")
```

```
for o in orders:
    if o not in kinds:
        kinds.append(o)
```

원하는 것은

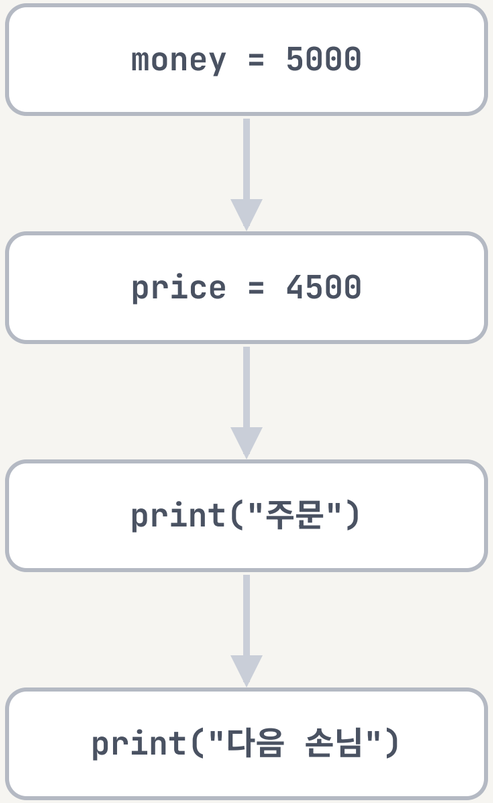
메뉴판에 있으면 가격을 찍고,
없으면 없다고 말해요.

조건에 따라 실행할 줄을 고르는 문법이 if예요.

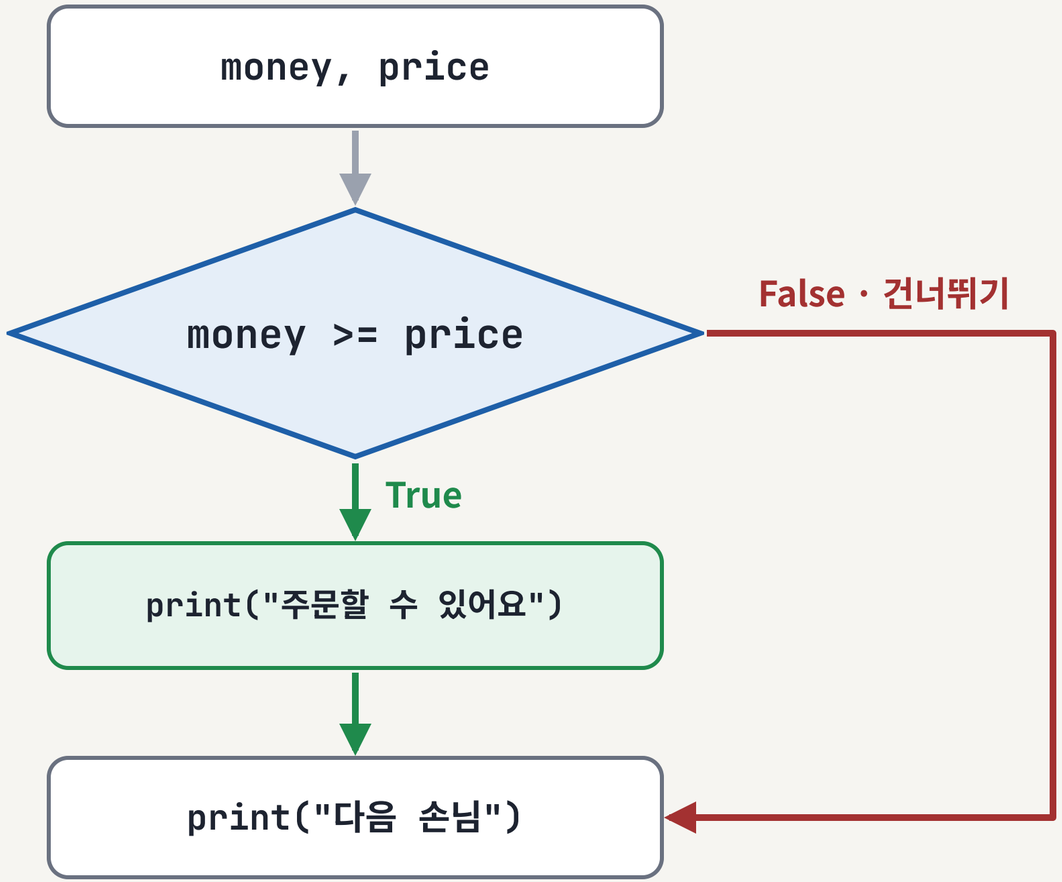
1. 참과 거짓으로 길 고르기

if는 코드에 갈림길을 만들어요

지금까지



if를 쓰면



마름모가 갈림길이에요. True면 아래로, False면 옆길로 가요.

1. 참과 거짓으로 길 고르기

가장 짧은 if는 조건 한 줄, 들여쓴 줄 하나예요

[2]

```python
money = 5000
price = 4500
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```

```
주문할 수 있어요
다음 손님
```

← 조건 · True나 False가 나와요

← 끝에 콜론

← 4칸 들여쓰면 True일 때만 실행

← if 밖이라 늘 실행

콜론 뒤에서 Enter를 치면 VS Code가 4칸을 넣어 줘요.

1. 참과 거짓으로 길 고르기

조건 자리에는 True나 False가 나오는 식을 적어요

[3]
```python
print(money >= price)
print(money == price)
print("tea" in menu)
print("tea" not in menu)
print(type(money >= price))
```

```
True
False
False
True
<class 'bool'>  ← True, False의 종류 이름
```

기호	뜻	배운 곳
==	같아요	12
!=	달라요	12
< >	작아요, 커요	12
<= >=	작거나 같아요, 크거나 같아요	12
in	들어 있어요	14
not in	안 들어 있어요	14

if는 조건이 True인지 False인지 보고 길을 골라요.

2. if, else, elif

조건이 False면 들여쓴 줄을 건너뛰어요

[1]
```
price = 4500
money = 5000
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```
Python

```
주문할 수 있어요
다음 손님
```

5000 >= 4500

True

← 실행해요

[2]
```
money = 3000
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```
Python

```
다음 손님
```
← 출력이 한 줄뿐이지만 오류가 아니에요

3000 >= 4500

False

← 건너뛰어요

같은 코드라도 값에 따라 실행하는 줄이 달라져요.

2. if, else, elif

들여쓴 줄까지가 if에 속해요

[3]
```python
money = 3000
if money >= price:
    print("주문할 수 있어요")
    print("커피 나왔어요")
print("다음 손님")
```

← if 안 · 둘 다 건너뛰어요

```
다음 손님
```

[4]
```python
money = 3000
if money >= price:
    print("주문할 수 있어요")
print("커피 나왔어요")
print("다음 손님")
```

← if 안 · 건너뛰어요

← 4칸을 빼먹어서 if 밖이에요

```
커피 나왔어요
다음 손님
```

← 돈이 모자라도 커피가 나와요

오류 없이 결과만 틀려요. 들여쓰기 칸 수가 if의 범위를 정해요.

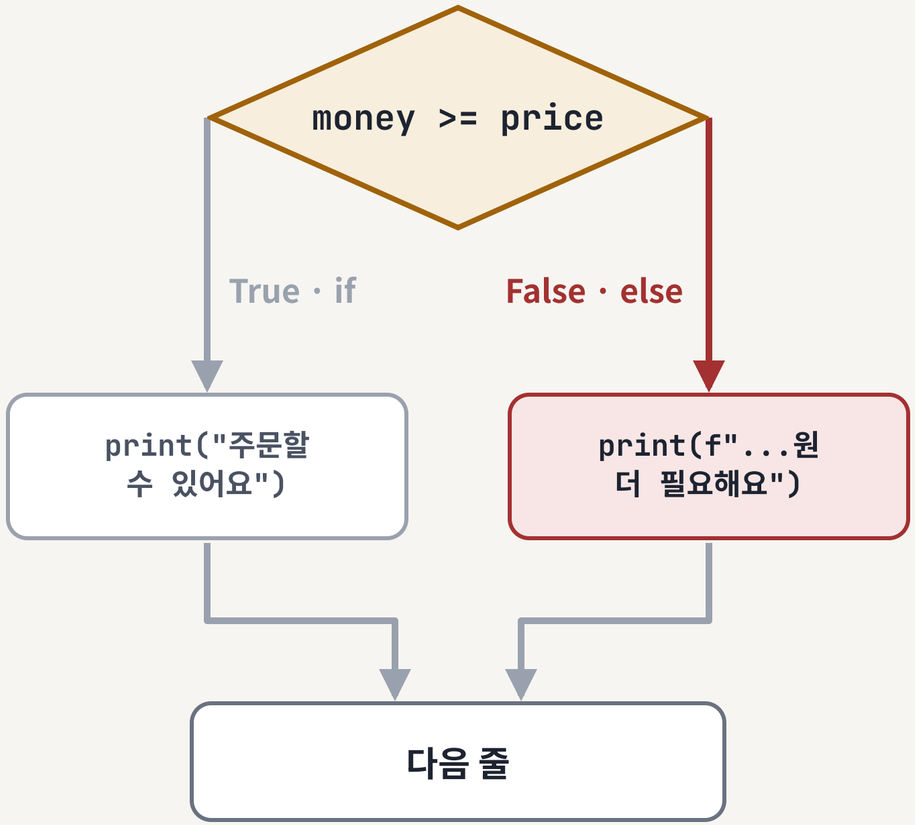
2. if, else, elif

else는 조건이 False일 때 갈 길이에요

[5]
```python
money = 3000
if money >= price:
    print("주문할 수 있어요")
else:
    print(f"{price - money}원 더 필요해요")
```

```
1500원 더 필요해요
```

else 뒤에는 조건 없이 콜론만 붙이고, if와 같은 칸에 맞춰요



if와 else 중 하나만, 그리고 반드시 하나는 실행해요.

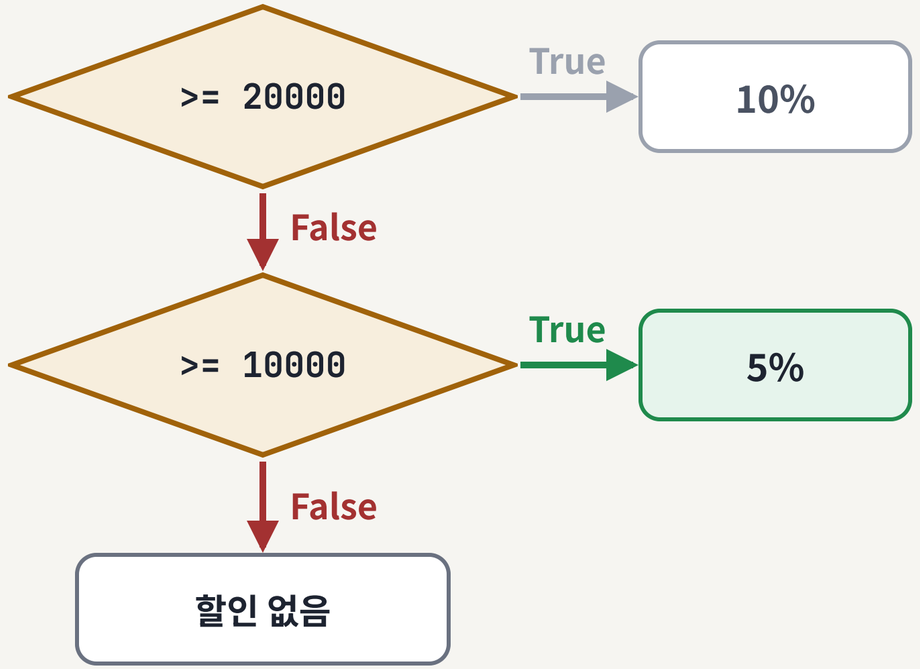
2. if, else, elif

elif로 갈림길을 셋 이상 만들어요

[6]

```python
total = 12000
if total >= 20000:                    # False
    print("10% 할인")
elif total >= 10000:                  # True · 실행
    print("5% 할인")
else:                                 # 보지 않음
    print("할인 없음")
```

```
5% 할인
```



total만 바꿔 실행	출력
`total = 25000`	10% 할인
`total = 12000`	5% 할인
`total = 3000`	할인 없음

위에서부터 묻고, 처음 True인 하나만 실행해요.

2. if, else, elif

조건 순서를 바꾸면 결과가 달라져요

[9]
```python
total = 25000
if total >= 10000:
    print("5% 할인")
elif total >= 20000:
    print("10% 할인")
else:
    print("할인 없음")
```

```
5% 할인
```

← 25000은 10000 이상이라 여기서 True

← 묻지도 않아요

← 25000원인데 5%예요

× ~~if total >= 10000:~~ → ✓ `if total >= 20000:` 좁은 조건을 위에

앞 장의 순서로 바꾸면 25000은 10% 할인, 12000은 5% 할인이 돼요.

범위가 겹치는 조건은 좁은 쪽부터 적어요.

2. if, else, elif

if를 따로 쓰면 둘 다 실행될 수 있어요

[10]
```python
total = 25000
if total >= 20000:
    print("10% 할인")
if total >= 10000:
    print("5% 할인")
```

```
10% 할인
5% 할인
```

← 갈림길 하나

← 갈림길 또 하나 · 다시 물어요

← 할인을 두 번 받아요

[7]
```python
total = 25000
if total >= 20000:
    print("10% 할인")
elif total >= 10000:
    print("5% 할인")
else:
    print("할인 없음")
```

```
10% 할인
```

← elif · 처음 True에서 멈춰요

하나만 고르면 elif, 각각 따로 확인하면 if 여러 개예요.

3. 조건 합치기

and는 두 조건이 모두 True일 때만 True예요

[1]
```python
height = 130
age = 9
if height >= 120 and age >= 7:
    print("탈 수 있어요")
else:
    print("다음에 타요")
```

```
탈 수 있어요
```

키 `130 >= 120` → `True`

나이 `9 >= 7` → `True`

and `True and True` → `True`

[2]
```python
height = 110
if height >= 120 and age >= 7:
    print("탈 수 있어요")
else:
    print("다음에 타요")
```

```
다음에 타요
```

키 `110 >= 120` → `False`

나이 `9 >= 7` → `True`

and `False and True` → `False`

12의 놀이기구 조건을 if에 넣어 실행할 줄을 골라요.

3. 조건 합치기

if 안의 if는 and 한 줄로 줄일 수 있어요

[3]
```python
height = 130
if height >= 120:
    if age >= 7:
        print("탈 수 있어요")
```

```
탈 수 있어요
```

← 4칸 · 바깥 if가 True일 때만 물어요

← 8칸 · 둘 다 True일 때만 실행

and로 줄이면

[ ]
```python
if height >= 120 and age >= 7:
    print("탈 수 있어요")
```

```
탈 수 있어요
```

← 셀 [1]과 같은 결과, 들여쓰기는 한 번

들여쓰기가 깊어지면 and로 합칠 수 있는지 먼저 봐요.

3. 조건 합치기

or는 하나만 True여도 True예요

[4]
```python
is_member = False
total = 60000
if is_member or total >= 50000:
    print("할인해 드려요")
```

```
할인해 드려요
```

회원 `is_member` → `False`

금액 `60000 >= 50000` → `True`

or `False or True` → `True`

앞	뒤	and	or
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

or가 False인 건 둘 다 False일 때뿐이에요

말로 '그리고'면 and, '또는'이면 or를 써요.

3. 조건 합치기

not과 not in으로 반대를 물어요

[5]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
order = "tea"
if not order in menu:                    # not (in)
    print("메뉴판에 없어요")
if order not in menu:                    # not in
    print("메뉴판에 없어요")
```

```
메뉴판에 없어요
메뉴판에 없어요
```

in `"tea" in menu` → `False`

not `not False` → `True`

결과는 같아요. 문장처럼 읽히는 not in을 더 많이 써요.

없을 때만 할 일이 있으면 else 대신 조건을 뒤집어요. 1파트 처음의 KeyError를 막는 첫 방법이에요.

"없으면"은 not in, "아니면"은 !=로 적어요.

3. 조건 합치기

범위는 9 <= hour < 18처럼 이어 적어요

영업시간 · 9시부터 18시 전까지



[6]
```python
hour = 14
if 9 <= hour < 18:                    # 줄인 방법
    print("영업 중이에요")
if hour >= 9 and hour < 18:           # 원래 방법
    print("영업 중이에요")
```

```
영업 중이에요
영업 중이에요
```

[7]
```python
hour = 20
if 9 <= hour < 18:
    print("영업 중이에요")
else:
    print("문 닫았어요")
```

```
문 닫았어요
```

"사이에 있나요?"는 작은 값 <= 변수 < 큰 값으로 물어요.

3. 조건 합치기

비어 있는 값은 False로 봐요

[8]
```python
print(bool(0), bool(4500))
print(bool(""), bool("tea"))
print(bool([]), bool(["tea"]))
print(bool({}), bool(set()))
print(bool(None))
```

```
False True
False True
False True
False False
False
```

종류	False로 봐요	True로 봐요
숫자	0	4500
문자열	""	"tea"
리스트	[]	["tea"]
딕셔너리, 집합	{} set()	{"tea"}
15의 None	None	

0과 빈 것, None만 False예요

bool(값)은 그 값을 if에 넣었을 때 True로 볼지 False로 볼지 알려 줘요.

3. 조건 합치기

if orders:는 하나라도 있는지 묻는 거예요

[9]

```python
orders = []
if orders:
    print(len(orders), "잔 주문")
else:
    print("아직 주문이 없어요")
if len(orders) > 0:
    print(len(orders), "잔 주문")
else:
    print("아직 주문이 없어요")
```

Python

```
아직 주문이 없어요
아직 주문이 없어요
```

← 줄인 방법 · 빈 리스트는 False

← 원래 방법 · 뜻이 더 잘 보여요

← 두 방법 모두 else로 가요

처음엔 len(orders) > 0으로 적어도 돼요. if orders:는 읽을 수 있으면 돼요.

4. for와 함께, 줄여 쓰기

for 안의 if로 골라 담아요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
prices = [4500, 5000, 6500, 4000]
cheap = []
for p in prices:
    if p <= 5000:
        cheap.append(p)
print(cheap)
```

if는 for 안 · 4칸

append는 if 안 · 8칸

Python

```
[4500, 5000, 4000]
```

prices

4500	5000	6500	4000
<= 5000	<= 5000	건너뜀	<= 5000

cheap

4500	5000	4000

True인 값만 append해요

빈 리스트, for, if, append가 골라 담기의 기본 모양이에요.

4. for와 함께, 줄여 쓰기

조건에 맞는 것만 세요

[2]
```python
orders = ["coffee", "latte", "coffee", "cake", "tea", "coffee"]
n = 0
for o in orders:
    if o == "coffee":
        n += 1
print(n)
print(orders.count("coffee"))
```
Python

True일 때만 1 더하기

```
3
3
```

← 같은 값 하나는 count로 바로 세요

o	== "coffee"	n
coffee	True	1
latte	False	1
coffee	True	2
cake	False	2
tea	False	2
coffee	True	3

세기는 n = 0에서 시작해서, 조건이 True일 때 n += 1이에요.

4. for와 함께, 줄여 쓰기

딕셔너리에서 조건에 맞는 메뉴를 찾아요

[3]
```python
for name, price in menu.items():
    if price <= 5000:
        print(name, price)
```

← 값 price로 물어요

```
coffee 4500
latte 5000
```

coffee	latte	cake
4500	5000	6500
<= 5000 → 찍어요	<= 5000 → 찍어요	6500 → 건너뛰어요

키로 물을 땐
if name == "cake":

for k, v in d.items(): 안에서 if로 키나 값을 물어요.

4. for와 함께, 줄여 쓰기

if와 else를 한 줄로 줄여 쓸 수 있어요

[4]
```python
total = 12000
if total >= 10000:                                          # 원래 방법 · 4줄
    msg = "5% 할인"
else:
    msg = "할인 없음"
print(msg)
msg = "5% 할인" if total >= 10000 else "할인 없음"          # 줄인 방법 · 1줄
print(msg)
```

```
5% 할인
5% 할인
```

```
msg = "5% 할인" if total >= 10000 else "할인 없음"
      ──────    ──────────────    ──────────────
      True일 때 값   조건          False일 때 값
```

변수에 둘 중 하나를 넣을 때만 줄여요.

4. for와 함께, 줄여 쓰기

15의 get을 이제 if로 풀어서 읽어요

[5]
```python
if "cakee" in menu:
    price = menu["cakee"]
else:
    price = 5000
print(price)
print(menu["cakee"] if "cakee" in menu else 5000)
print(menu.get("cakee", 5000))
```

```
5000
5000
5000
```

← 세 방법 모두 5000

← 1 · if, else 4줄 · 이번 챕터

← 2 · 한 줄 if else · 15 참고 칸

← 3 · get · 15

get은 위의 if, else를 대신해 주는 메서드예요. 실제로는 get을 써요.

4. for와 함께, 줄여 쓰기

16에서 읽기만 한 코드를 직접 쳐 봐요

discard = 있으면 remove

[6]
```python
menus = {"coffee", "cake"}
if "juice" in menus:                    # 원래 방법
    menus.remove("juice")
print(sorted(menus))
menus.discard("juice")                  # 줄인 방법
print(sorted(menus))
```

```
['cake', 'coffee']
['cake', 'coffee']
```

set() = 없을 때만 append

[7]
```python
kinds = []                              # 원래 방법
for o in orders:
    if o not in kinds:
        kinds.append(o)
print(kinds)
print(sorted(set(orders)))              # 줄인 방법
```

```
['coffee', 'latte', 'cake', 'tea']
['cake', 'coffee', 'latte', 'tea']
```

리스트는 처음 나온 순서, 집합은 sorted() 순서예요

16의 "참고, 읽기만" 칸은 이제 모두 직접 칠 수 있어요.

5. 자주 나는 오류

if 뒤에는 = 하나가 아니라 == 두 개예요

[1]
```
order = "coffee"
if order = "coffee":
    print("커피예요")
```
Python

```
Cell In[1], line 2
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

← 둘째 줄이 문제예요

"== 를 쓰려던 거 아니에요?" · := 는 배우지 않은 기호예요

✕ ~~`if order = "coffee":`~~ → ✓ `if order == "coffee":`

= 하나는 넣기, == 두 개는 묻기 (12)

SyntaxError는 실행 전에 문법을 검사하다 나서, 셀의 어느 줄도 실행되지 않아요.

SyntaxError가 나면 line 숫자의 줄부터 봐요.

5. 자주 나는 오류

콜론과 들여쓰기를 빠뜨리면 실행 전에 멈춰요

[2]
```python
if order == "coffee"
    print("커피예요")
```
```
SyntaxError: expected ':'
```

콜론이 없어요

if 줄 끝에 :

[3]
```python
if order == "coffee":
print("커피예요")
```
```
IndentationError: expected an indented block after 'if' statement on line 1
```

if 안에 들여쓴 줄이 없어요

print 앞에 4칸

[4]
```python
if order == "coffee":
    print("커피예요")
      print("나왔어요")
```
```
IndentationError: unexpected indent
```

들여쓰기 칸 수가 달라요

같은 칸(4칸)에 맞추기

IndentationError는 들여쓰기 오류예요. 칸 수를 4칸에 맞춰요.

5. 자주 나는 오류

or 뒤에도 비교를 다시 적어요

[5]
```python
order = "cake"
if order == "coffee" or "latte":
    print("커피 종류예요")
print(order == "coffee" or "latte")
```

```
커피 종류예요
latte
```

← cake인데 커피 종류래요

앞 `order == "coffee"` → `False`

or `False or "latte"`

결과 `"latte"` ← 빈 글자가 아니라 True로 봐요

[6]
```python
if order == "coffee" or order == "latte":
    print("커피 종류예요")
else:
    print("커피 종류가 아니에요")
print(order in ["coffee", "latte"])
```

```
커피 종류가 아니에요
False
```

← 비교를 둘 다 적거나

← in과 리스트로 물어요

말로는 "coffee 또는 latte"지만 코드는 order == 를 두 번 적어요.

5. 자주 나는 오류

글자와 숫자는 크기를 비교할 수 없어요

[7]
```
price = "4500"
if price >= 4000:
    print("4000원 넘어요")
```
Python

← 따옴표가 있으면 글자(str)예요

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

비교한 두 값의 종류

[8]
```
if int(price) >= 4000:
    print("4000원 넘어요")
```
Python

← 12의 int()로 숫자로 바꿔요

```
4000원 넘어요
```

== 로 비교하면 오류 없이 False가 나와서 if가 조용히 건너뛰어요. 더 찾기 어려워요.

비교 오류가 나면 type()으로 두 값의 종류부터 봐요.

6. 실습

장바구니 금액에 따라 할인해요

[1]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = {"coffee": 2, "cake": 1}
total = 0
for name, qty in cart.items():                    # 15 · 합계
    total += menu[name] * qty
if total >= 20000:                                # elif · 할인율
    rate = 10
elif total >= 10000:
    rate = 5
else:
    rate = 0
sale = total * rate // 100                        # 12 · 몫 //
print(f"합계 {total:>7,}원")
print(f"할인 {sale:>7,}원 ({rate}%)")
print(f"결제 {total - sale:>7,}원")
```

Python

```
합 계   15,500원
할 인      775원 (5%)
결 제   14,725원
```

for로 합계를 내고, elif로 할인율 하나를 골라요.

계산 따라가기

coffee 2잔 4,500 × 2 = 9,000

cake 1개 6,500 × 1 = 6,500

합계 15,500 · 10000 이상이라 5%

할인 15500 × 5 // 100 = 775

6. 실습

점수로 등급을 매겨요

[2]
```python
scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
for name, score in scores:
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    else:
        grade = "F"
    print(f"{name:<5}{score:>3}  {grade}")
```

elif 계단 · 8칸 안

for 안, if 밖 · 4칸

Python

```
kim   92  A
lee   78  C
park  85  B
choi  59  F
```

A 90 이상

B 80 이상

C 70 이상

F 나머지

높은 점수 조건부터 내려가요

for 안에 elif 계단을 넣으면 한 명씩 등급이 나와요.

6. 실습

메뉴판에 없는 주문은 빼고 계산해요

[3]
```python
orders = ["coffee", "tea", "cake", "coffee", "juice"]
total = 0
skipped = []
for o in orders:
    if o in menu:
        print(f"{o:<8}{menu[o]:>6,}원")
        total += menu[o]
    else:
        skipped.append(o)
print(f"합계 {total:,}원")
if skipped:
    print("준비 중:", skipped)
```

Python

```
coffee    4,500원
cake      6,500원
coffee    4,500원
합계 15,500원
준비 중: ['tea', 'juice']
```

← 있으면 찍고 합계에 더해요

← 없으면 따로 모아요 · KeyError 없음

← 하나라도 있을 때만 알려 줘요

← 1파트 처음에 멈췄던 tea예요

if는 KeyError가 날 줄 앞에서 먼저 물어 막아 줘요.

6. 실습

어떤 모양을 쓸지 이렇게 골라요

모양	언제	예
`if`	할 일이 한 가지, 조건이 맞을 때만	있으면 알려 주기
`if, else`	둘 중 하나는 꼭	살 수 있음, 돈이 모자람
`if, elif, else`	셋 이상 중 하나만	할인율, 등급
`if 여러 개`	각각 따로 확인	쿠폰 여러 장
`and, or, not`	조건 여러 개를 하나로	키와 나이, 회원이거나 5만 원
`값1 if 조건 else 값2`	변수에 둘 중 하나 넣기	msg 고르기

처음에는 길게 적고, 같은 모양이 반복되면 줄여 써요.

정리

막히면 오류의 마지막 줄을 봐요

1	`SyntaxError: ... Maybe you meant '=='`	if 뒤에 = 하나를 썼어요	`==`
2	`SyntaxError: expected ':'`	if, elif, else 줄 끝에 콜론이 없어요	끝에 `:`
3	`IndentationError: expected an indented block`	if 다음 줄을 들여쓰지 않았어요	4칸 들여쓰기
4	`IndentationError: unexpected indent`	들여쓰기 칸 수가 서로 달라요	같은 칸에 맞추기
5	`TypeError: '>=' not supported ... 'str' and 'int'`	글자와 숫자를 비교했어요	`int(price)`
6	오류는 없는데 조건이 늘 True	or 뒤에 비교 없이 값만 적었어요	`o == "a" or o == "b"`
7	오류는 없는데 결과가 이상함	elif 순서, 들여쓰기 범위를 봐요	좁은 조건부터

오류가 안 나도 결과가 이상하면 if 안에 print를 넣어 어느 줄이 실행됐는지 봐요.

한 장 요약

`if 조건:`	True일 때만 실행	`a or b`	하나라도 True
`else:`	False일 때 갈 길	`x not in menu`	없으면
`elif 조건:`	위에서부터 처음 True 하나	`9 <= h < 18`	범위 묻기
`    (4칸)`	들여쓴 줄까지 if 안	`if orders:`	비어 있지 않으면
`a and b`	둘 다 True	`값1 if 조건 else 값2`	둘 중 하나를 넣기

조건 뒤에 콜론, 다음 줄은 4칸. 묻기는 ==, 넣기는 =

다음 18에서는 while로 조건이 True인 동안 반복하고, break로 멈춰요.